Program 1: Basic Data Types

Scenario: Store student roll number, grade, and fee amount.

Executable Code:

```
#include <iostream>
using namespace std;
int main() {
   int roll = 101;
   char grade = 'A';
   float fee = 12500.50;
   cout << "Roll No: " << roll << endl;
   cout << "Grade: " << grade << endl;
   cout << "Fee: " << fee << endl;
   return 0;
}
```

Line-by-line Explanation:

- iostream is used for input/output.
- int stores whole numbers.
- char stores one character.
- float stores decimal numbers.
- cout displays values.
- return 0 ends the program.

Expected Output:

```
Roll No: 101
Grade: A
Fee: 12500.5
```

Viva Questions:

- What is a basic data type?
- Why is char used for grade?
- What is the use of cout?

Program 2: if-else

Scenario: Check whether a student has passed or failed.

Executable Code:

```
#include <iostream>
using namespace std;
int main() {
   int marks = 45;
   if (marks >= 40) {
      cout << "Pass";
```

```
    } else {
        cout << "Fail";
    }
    return 0;
}
```

Line-by-line Explanation:

- marks stores student score.
- if checks pass condition.
- else executes when condition is false.
- cout prints result.

Expected Output:

Pass

Viva Questions:

- What is selection?
- What happens if marks are 35?
- What does else do?

Program 3: Loop and Array

Scenario: Print marks of five students.

Executable Code:

```
#include <iostream>
using namespace std;
int main() {
    int marks[5] = {78, 82, 91, 67, 88};
    for (int i = 0; i < 5; i++) {
        cout << marks[i] << " ";
    }
    return 0;
}
```

Line-by-line Explanation:

- marks array stores five values.
- for loop repeats five times.
- i is the index variable.
- marks[i] accesses each element.

Expected Output:

78 82 91 67 88

Viva Questions:

- What is an array?
- Why start index from 0?
- Why use loop here?

Program 4: Functions

Scenario: Create an addition function for reuse.

Executable Code:

```
#include <iostream>
using namespace std;
int add(int, int);
int main() {
   int a = 10, b = 20;
   cout << "Sum = " << add(a, b) << endl;
   return 0;
}
int add(int x, int y) {
   return x + y;
}
```

Line-by-line Explanation:

- Prototype informs compiler.
- main starts execution.
- add(a, b) calls function.
- x and y receive values.
- return gives output.

Expected Output:

Sum = 30

Viva Questions:

- Why use functions?
- What is a prototype?
- What is return value?

Program 5: Class and Object

Scenario: Store student details using class and object.

Executable Code:

```
#include <iostream>
using namespace std;
class Student {
public:
   string name;
```

```
    int age;
    void show() {
        cout << name << " " << age << endl;
    }
};
int main() {
    Student s1;
    s1.name = "Amit";
    s1.age = 20;
    s1.show();
    return 0;
}
```

Line-by-line Explanation:

- class defines blueprint.
- string and int are data members.
- show() is a member function.
- s1 is an object.
- Values are assigned using dot operator.

Expected Output:

Amit 20

Viva Questions:

- What is a class?
- What is an object?
- What is the dot operator used for?

Program 6: Constructor and Destructor

Scenario: Show automatic object initialization and cleanup.

Executable Code:

```
#include <iostream>
using namespace std;
class Demo {
public:
    Demo() {
        cout << "Constructor called
";
    }
    ~Demo() {
        cout << "Destructor called
";
    }
```

```
};
int main() {
   Demo d;
   return 0;
}
```

Line-by-line Explanation:

- Demo() is constructor.
- It runs when object is created.
- ~Demo() is destructor.
- It runs when object ends.
- Messages show execution order.

Expected Output:

```
Constructor called
Destructor called
```

Viva Questions:

- What is constructor?
- When is destructor called?
- Why is it useful?

Program 7: Static Member

Scenario: Count how many objects are created.

Executable Code:

```
#include <iostream>
using namespace std;
class Student {
public:
   static int count;
   Student() {
      count++;
   }
};
int Student::count = 0;
int main() {
   Student s1, s2, s3;
   cout << Student::count;
   return 0;
}
```

Line-by-line Explanation:

- static count belongs to class.

- constructor increases count.
- Student::count defines shared data.
- class name accesses static member.

Expected Output:

3

Viva Questions:

- What is a static member?
- Why is it shared?
- How to access it?

Program 8: Inline and Friend Function

Scenario: Access private data using inline getter and friend function.

Executable Code:

```
#include <iostream>
using namespace std;
class Test {
private:
   int value;
public:
   Test(int v) {
      value = v;
   }
   inline int getValue() {
      return value;
   }
   friend void show(Test t);
};
void show(Test t) {
   cout << t.value;
}
int main() {
   Test obj(50);
   cout << obj.getValue() << endl;
   show(obj);
   return 0;
}
```

Line-by-line Explanation:

- value is private.
- constructor stores initial value.
- inline getter returns value quickly.

- friend function accesses private data.

Expected Output:

50
50

Viva Questions:

- What is inline function?
- What is friend function?
- Why use friend function?